Alzheimer Maths

Yami

1 Foundations of the System (Axioms)

1.1 The Initial Set

The universe is initialized with a finite inventory containing the integers $[1, \ldots, 9]$, their negatives $[-1, \ldots, -9]$, and their reciprocal fractions $[1, \ldots, 1/9]$.

The 1 here is considered a different iteration from 1, but acts as 1.

And their negative reciprocal fractions.

1.2 Axiom 1: Destructive Consumption

Every mathematical operation permanently consumes its operands. A number that is used is destroyed and no longer exists, unless it is recreated by the result of an operation.

The available operations are multiplication and addition, and they act as they do under the ZFC system.

1.3 Axiom 2: Multiplicity

The same number can exist in several distinct iterations (tokens) within the inventory.

1.4 Axiom 3: Operation P (Loss)

The operation $P(x)$ consumes the token x without generating a new result. It is pure destruction.

1.5 Axiom 4: Operation R (Reset)

The operation R clears the current inventory and resets the system exactly to its initial state.

1.6 Axiom 5

A term, if it has only one iteration, cannot be used twice in the same operation.

For example, if the set contains only $[1, 0]$, the operation

$$1 - 1$$

is forbidden.

2 Fundamental Theorems

2.1 Theorem 1: The Number of Numbers

"There will always be fewer numbers in the system than at the beginning."

Proof. Each binary operation ($+$, $\times$, $-$, $\div$) consumes two tokens to generate only one (net change: -1 token).

Each operation P consumes one token and generates zero (net change: -1 token).

The total number of tokens strictly decreases at every step.

2.2 Theorem 2: The End of the World

"If you start with a set of N numbers, you will have at most N operations in this system, and the N-th operation different from R will always be a P."

Proof. According to Theorem 1, each operation removes exactly one token from the system.

After $N-1$ operations, the inventory contains only one token:

$$N - (N-1) = 1.$$

At this point, binary operations are impossible due to a lack of operands. The only remaining legal action different from R is the unary operation P.

The N-th operation that is not an R is therefore inevitably an extinction through P.

2.3 Theorem 3: Maximum Possible Number

Knowing that, for two positive integers a and b different from 1, we have

$$a * b \geq a + b,$$

the maximum possible number in our set is

$$9! + \text{an even number of negative factors multiplied together, greater than 1.}$$

Here, we can use at most eight negative factors:

$$(-9)(-8)(-7)(-6)(-5)(-4)(-3)(-2) = 9!.$$

We therefore have

$$(9!)^2.$$

And we can add to this number the sum of the decimals, from 1 to $\frac{1}{9}$:

$$(9!)^2 \left(1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \frac{1}{6} + \frac{1}{7} + \frac{1}{8} + \frac{1}{9}\right)^2.$$

Let us call this number, for simplicity, Max.

Any other method is non-optimal. Adding the negative terms reduces the value, multiplying the fractions reduces the value, and adding positive values creates numbers smaller than multiplying them.

This considers the set of possible strategies, taking each group separately.

For the numbers from 1 to 9, we can add or multiply them. Since

$$a * b \geq a + b,$$

multiplying them creates a larger number.

For negative numbers, we can add or multiply them. Adding them creates more negative numbers. Even if we try to add them and then multiply them by -1, we will create a number smaller than a large multiplication.

To create an even larger number with the first 9!, the optimal choice is to multiply them again.

For positive fractions, knowing that they are between -1 and 1, multiplying them will always create smaller numbers. Adding them is the optimal choice.

Their sum will be greater than 1, so multiplication is then the optimal choice.

For negative fractions, the strategy follows the same logic as for fractions: a sum is used as a term in a product. To transform the negative sum into a positive term in a product, we use the -1 left over from the negative terms.

The set of terms created is greater than 1. The optimal way to combine them is therefore multiplication.

We obtain:

$$\boxed{\text{Max} = (9!)^2 \left(1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \frac{1}{6} + \frac{1}{7} + \frac{1}{8} + \frac{1}{9}\right)^2}$$

Since

$$1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \frac{1}{6} + \frac{1}{7} + \frac{1}{8} + \frac{1}{9} = \frac{7129}{2520},$$

we have

$$\boxed{\text{Max} = (362880)^2 \left(\frac{7129}{2520}\right)^2}$$

Numerically,

$$\boxed{\text{Max} \approx 1.0538 \times 10^{12}}$$

3 Python Simulation

3.1 Checking Token Multiplicity

```python
def T2valide(t1, t2, nombres):
    if t1 != t2:
        return True
    else:
        n = 0
        for i in nombres:
            if i == t1:
                n += 1
```

```
        if n > 1:
            return True

    return False
```

3.2 Simulation Program

```
print("Type stop to end the simulation")

Run = True
New_world = True


def T2valide(t1, t2, nombres):
    if t1 != t2:
        return True

    n = 0

    for i in nombres:
        if i == t1:
            n += 1

    return n > 1


while Run:

    if New_world:
        nombres = [
            # Positive integers
            1.0, 2.0, 3.0, 4.0, 5.0,
            6.0, 7.0, 8.0, 9.0,

            # Negative integers
            -1.0, -2.0, -3.0, -4.0, -5.0,
            -6.0, -7.0, -8.0, -9.0,

            # Positive fractions
            1.0, 1/2, 1/3, 1/4, 1/5,
            1/6, 1/7, 1/8, 1/9,

            # Negative fractions
            -1.0, -1/2, -1/3, -1/4, -1/5,
            -1/6, -1/7, -1/8, -1/9
        ]

        New_world = False
        print("New inventory:", nombres)
```

```
43
44      else:
45          operation = input(
46              "Operation to perform (+, *, P, R, stop): "
47          )
48
49          if operation == "stop":
50              Run = False
51
52          elif operation == "R":
53              New_world = True
54
55          elif operation == "P":
56              T1 = float(input("Number to destroy: "))
57
58              if T1 in nombres:
59                  nombres.remove(T1)
60                  print("Inventory:", nombres)
61              else:
62                  print("Invalid operation: token not found.")
63
64          elif operation in ("+", "*"):
65              T1 = float(input("First number: "))
66
67              if T1 not in nombres:
68                  print("First token not found.")
69                  continue
70
71              T2 = float(input("Second number: "))
72
73              if T2 not in nombres or not T2valide(
74                  T1, T2, nombres
75              ):
76                  print("Invalid operation: tokens unavailable.")
77                  continue
78
79              # Consume both tokens
80              nombres.remove(T1)
81              nombres.remove(T2)
82
83              # Create the result
84              if operation == "+":
85                  resultat = T1 + T2
86              else:
87                  resultat = T1 * T2
88
89              nombres.append(resultat)
90
91              print("Result:", resultat)
92              print("Inventory:", nombres)
93
```

```
94            else:
95                print("Unsupported operation.")
```